INSTITUIÇÃO DE ENSINO SUPERIOR

CURSO DE COMPUTAÇÃO / SISTEMAS DE INFORMAÇÃO

LUAN FEITOSA SANTOS
JOSÉ ÍTALO S. C. DANTAS
MARCELO VITOR VIANA DA SILVA
LETICIA VIVIANE PEREIRA DA SILVA

STOCK4EVENT MOBILE

Documentação técnica do aplicativo de gestão de estoque e compras B2B no setor de bebidas

2026

LUAN FEITOSA SANTOS
JOSÉ ÍTALO S. C. DANTAS
MARCELO VITOR VIANA DA SILVA
LETICIA VIVIANE PEREIRA DA SILVA

STOCK4EVENT MOBILE

Documentação técnica do aplicativo de gestão de estoque e compras B2B no setor de bebidas

Documentação apresentada como registro do desenvolvimento do cliente móvel do ecossistema Stock4event, descrevendo arquitetura, fluxos de uso, integração com a API REST e organização do código-fonte.

2026

RESUMO

Este documento descreve o aplicativo Stock4event Mobile, interface cliente do sistema Stock4event, construída com Expo, React Native e TypeScript. O software atende dois modos de uso após o cadastro: o perfil de empresa (fornecedor), voltado à operação de estoque, quiosques, marketplace interno e logística; e o perfil de cliente, voltado à descoberta de quiosques, montagem de pedido, pagamento e acompanhamento de entregas. A comunicação com o backend ocorre por HTTP/JSON na porta 8081. A sessão é persistida localmente; o carrinho de compras e o perfil de uso (Cliente, Fornecedor ou ambos) também ficam no dispositivo. O texto organiza a arquitetura em camadas, os fluxos de autenticação e de negócio, o catálogo de telas, os serviços de API e as decisões de interface. Não se pretende substituir o contrato da API nem a modelagem física do banco, ambos mantidos no repositório do backend.

Palavras-chave: React Native; Expo; comércio B2B; gestão de estoque; aplicativo móvel.

ABSTRACT

This document describes the Stock4event Mobile application, the client interface of the Stock4event system, built with Expo, React Native and TypeScript. After registration the software supports two usage modes: a supplier/company profile for stock, kiosks, internal marketplace and logistics; and a customer profile for discovering stores, assembling orders, paying and tracking deliveries. The app talks to the backend over HTTP/JSON on port 8081. Session data, the shopping cart and the usage profile are stored on the device. The text covers layered architecture, authentication and business flows, screens, API services and UI decisions.

Keywords: React Native; Expo; B2B commerce; inventory management; mobile application.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura em camadas do aplicativo e da API

Figura 2 – Encadeamento dos provedores de contexto

Figura 3 – Fluxo de autenticação e entrada no aplicativo autenticado

Figura 4 – Abas do perfil cliente

Figura 5 – Etapas de acompanhamento do pedido

LISTA DE TABELAS

Tabela 1 – Pacotes principais do projeto

Tabela 2 – Contextos globais

Tabela 3 – Telas do fluxo não autenticado

Tabela 4 – Telas do perfil empresa

Tabela 5 – Telas do perfil cliente

Tabela 6 – Endpoints consumidos pelo aplicativo

Tabela 7 – Chaves de armazenamento local

SUMÁRIO

1 INTRODUÇÃO

1.1 Objetivo geral

1.2 Objetivos específicos

1.3 Justificativa

1.4 Escopo e limitações

1.5 Organização do texto

2 TECNOLOGIAS E ORGANIZAÇÃO DO CÓDIGO

2.1 Stack

2.2 Estrutura de diretórios

2.3 Convenções de implementação

3 ARQUITETURA

3.1 Visão em camadas

3.2 Provedores e navegação raiz

3.3 Resolução da URL da API

4 AUTENTICAÇÃO, CADASTRO E SESSÃO

4.1 Telas iniciais

4.2 Perfis de cadastro

4.3 Sessão e perfil de uso

5 OPERAÇÃO DA EMPRESA (FORNECEDOR)

6 EXPERIÊNCIA DO CLIENTE

6.1 Navegação inferior

6.2 Compra e carrinho

6.3 Pedidos e conta

7 SERVIÇOS E INTEGRAÇÃO COM A API

8 PERSISTÊNCIA NO DISPOSITIVO

9 INTERFACE GRÁFICA

10 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

APÊNDICE A – Como executar o projeto

1 INTRODUÇÃO

O atacado de bebidas opera com reposição frequente, múltiplos pontos de venda e necessidade de conferir pedido, endereço e pagamento sem recorrer a planilhas paralelas. O Stock4event reúne, no mesmo aplicativo, a rotina de quem vende (cadastro de produto, quiosque, estoque e logística) e a rotina de quem compra (catálogo, carrinho, checkout e histórico). O cliente móvel não implementa regras de negócio de servidor: ele apresenta estados, valida formulários e dispara as rotas já existentes da API.

1.1 Objetivo geral

Documentar o aplicativo Stock4event Mobile de modo que um leitor técnico consiga compreender o recorte de cada perfil, a origem dos dados e o caminho de uma ação de tela até o endpoint correspondente.

1.2 Objetivos específicos

a) descrever a stack Expo 54 / React Native 0.81 / TypeScript e a divisão em screens, services e contexts; b) registrar o fluxo de autenticação, inclusive escolha de perfil (Cliente, Fornecedor, Cliente e fornecedor); c) distinguir a navegação da empresa da navegação do cliente; d) listar os serviços HTTP e o armazenamento local (sessão, carrinho, PIX, perfil de uso); e) explicitar decisões de interface (navy e dourado no cliente e na autenticação; dourado e creme na operação da empresa).

1.3 Justificativa

Sem um mapa do front-end, o backend e o aplicativo divergem com rapidez: telas novas passam a inventar contratos, e telas antigas deixam de refletir o status real do pedido. A documentação reduz esse descompasso e serve de base para apresentação acadêmica e para onboarding de quem entra no repositório.

1.4 Escopo e limitações

O recorte é o repositório do aplicativo (pasta `src/` e configuração Expo). O servidor Spring Boot e o esquema PostgreSQL são citados apenas na medida em que o cliente os consome. Ficam fora: publicação em lojas, pipeline de CI, suíte formal de testes automatizados e o detalhamento jurídico de LGPD. Há um atalho de desenvolvimento com o usuário `admin` e senha `1234`, que não deve existir em ambiente de produção.

1.5 Organização do texto

A seção 2 apresenta tecnologias e pastas. A seção 3 trata da arquitetura. A seção 4 cobre login e cadastro. As seções 5 e 6 separam empresa e cliente. As seções 7 e 8 descrevem API e storage. A seção 9 sintetiza o visual. A seção 10 encerra. O apêndice A indica como executar o projeto localmente.

2 TECNOLOGIAS E ORGANIZAÇÃO DO CÓDIGO

2.1 Stack

O empacotamento é feito pelo Expo SDK 54. A interface usa React 19 e React Native 0.81, com suporte a web (`react-native-web`), o que explica a necessidade de diálogos próprios no lugar de `Alert.alert` em fluxos críticos, como o encerramento de sessão. A navegação é Native Stack (React Navigation 7). Ícones vêm de `@expo/vector-icons`. Gradientes usam `expo-linear-gradient`. Imagens de produto passam por `expo-image-picker` no cadastro da empresa. Não há Redux: o estado compartilhado está em Context API.

Tabela 1 – Pacotes principais do projeto

Pacote	Função
expo ~54	CLI, fontes, splash, constantes
react-native 0.81	componentes nativos e web
@react-navigation/native-stack	pilha de telas
@react-native-async-storage/async-storage	sessão, carrinho, PIX, perfil
expo-linear-gradient	fundos de cabeçalho
typescript ~5.9	tipagem estática

Fonte: Elaborado pelos autores.

2.2 Estrutura de diretórios

```
src/
  components/   UI reutilizável (botão, input, headers, auth, layout)
  config/       API_BASE_URL e montagem de URL de imagem
  context/      Auth, produtos, quiosque, carrinho, diálogo de confirmação
  hooks/        useAppGoBack e auxiliares de área segura
  navigation/   AuthNavigator, MainNavigator, tipos da pilha
  screens/      uma pasta por tela (hook + estilos quando o volume exige)
  services/     fetch tipado por domínio
  theme/        cores da empresa e da autenticação
  types/        contratos alinhados à API
  utils/        máscaras de CEP, CNPJ, cartão, data
```

Telas volumosas separam apresentação e regra: por exemplo, `useSacola.ts` concentra checkout, enquanto `SacolaScreen.tsx` só renderiza. No cliente, o mesmo padrão aparece em `usePedidos.ts` e `useExplorar.ts`.

2.3 Convenções de implementação

O front-end não cria rotas novas no servidor. Quando uma tela precisa de dado que a API ainda não oferece (chaves PIX locais, perfil de uso, itens do carrinho antes do POST da solicitação), o valor fica no AsyncStorage. Mensagens de erro de rede são normalizadas em `http.ts`. Datas e preços usam formatação pt-BR.

3 ARQUITETURA

3.1 Visão em camadas

A tela nunca chama `fetch` diretamente, salvo exceções históricas. O caminho padrão é componente → hook ou context → função em `services/` → `parseResponse`. Isso isola mudança de contrato (nome de campo, código HTTP) em um único arquivo por domínio.

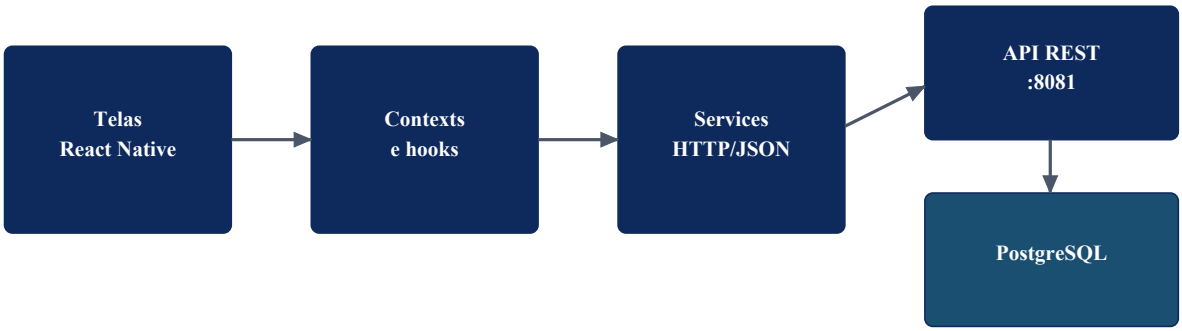


Figura 1 – Arquitetura em camadas do aplicativo e da API

Fonte: Elaborado pelos autores.

3.2 Provedores e navegação raiz

`App.tsx` envolve a árvore com Gesture Handler, Safe Area, diálogo de confirmação e autenticação. `AppNavigator` espera a hidratação da sessão. Sem token válido, monta `AuthNavigator`. Com sessão, envolve `MainNavigator` em `PurchaseCartProvider`, e este por sua vez em `ProductsProvider` e `QuiosqueProvider`. O carrinho, portanto, só existe depois do login; ao sair da conta, o

provedor é desmontado.

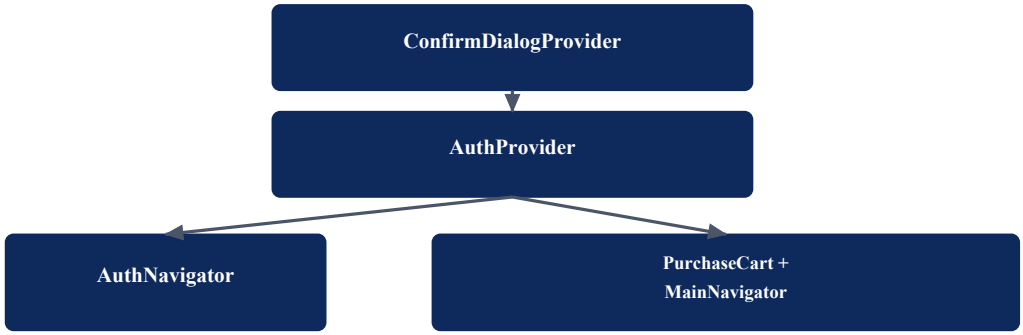


Figura 2 – Encadeamento dos provedores de contexto

Fonte: Elaborado pelos autores.

Tabela 2 – Contextos globais

Contexto	Responsabilidade
AuthContext	usuário, token, perfilUso, signIn, signOut, updateUser
PurchaseCartContext	itens da sacola, grupos por fornecedor, total
ProductsContext	lista de produtos da empresa logada
QuiosqueContext	barracas/quiosques da empresa
ConfirmDialogContext	modal de confirmação (web e nativo)

Fonte: Elaborado pelos autores.

3.3 Resolução da URL da API

`src/config/api.ts` monta `http://{host}:8081`. Em desenvolvimento, o host vem do `hostUri` do Expo; no emulador Android usa-se 10.0.2.2; nos demais casos, localhost. Imagens relativas da API são prefixadas por essa base em `getImageUrl`. Dispositivo físico na mesma Wi-Fi precisa do IPv4 da máquina, não de localhost.

4 AUTENTICAÇÃO, CADASTRO E SESSÃO

4.1 Telas iniciais

A pilha não autenticada começa em `Animation`: uma fase mostra carrinho e barra de progresso; a seguinte exibe a marca e o slogan. Em seguida vem Welcome, com Entrar e Criar conta. Login envia e-mail e senha a `POST /api/usuarios/login`. Esqueci a senha chama `POST /api/usuarios/recuperar-senha`. Criar conta abre a escolha

de perfil e só então o formulário de `POST /api/cadastro`.

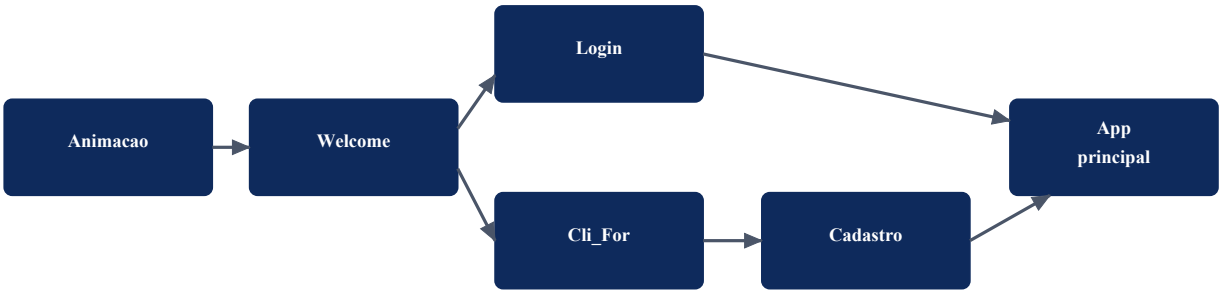


Figura 3 – Fluxo de autenticação e entrada no aplicativo autenticado

Fonte: Elaborado pelos autores.

Tabela 3 – Telas do fluxo não autenticado

Rota	Arquivo	Papel
Animation	AnimationScreen.tsx	Splash e carregamento inicial
Welcome	WelcomeScreen.tsx	Entrar ou criar conta
Login	Login.tsx	Credenciais
ForgotPassword	ForgotPasswordScreen.tsx	Redefinição de senha
Cli_For	Cli_For.tsx	Escolha de perfil
Register	RegisterScreen.tsx	Dados pessoais e da empresa

Fonte: Elaborado pelos autores.

4.2 Perfis de cadastro

O tipo `PerfilCadastro` admite Cliente, Fornecedor e ClienteFornecedor. Cliente envia nome, CPF (gravado no campo de documento da empresa no payload atual), telefone, e-mail e senha. Fornecedor envia razão social, CNPJ, telefone, responsável, e-mail e senha. O perfil combinado pede os dois conjuntos. Após o cadastro, `savePerfilUso` associa o e-mail ao perfil escolhido no AsyncStorage, para que o próximo login abra a casca visual correta.

4.3 Sessão e perfil de uso

O token e o objeto `UsuarioLogado` ficam em `@stock4event_session`, com `expiresAt`. Na abertura, `AuthContext` tenta `GET /api/usuarios/me`; se o token for legado (`legacy:`), a sessão local é reaplicada sem essa chamada. `signOut` remove o

registro e zera usuário, token e perfil mesmo se o storage falhar. O encerramento no perfil do cliente usa o modal de `ConfirmDialog`, porque o alerta nativo não confirma ação de forma confiável no Expo Web.

5 OPERAÇÃO DA EMPRESA (FORNECEDOR)

Quando `perfilUso` não é Cliente, `HomeScreen` permanece na casca dourada. A barra inferior leva a início, quiosques, marketplace (Cart), cadastro de produto (botão central), sacola, carteira e cartões. O menu hamburger abre configurações, formas de pagamento e saída. A Home resume estoque e atalhos financeiros (`/api/financeiro/resumo`).

Tabela 4 – Telas do perfil empresa

Rota	Função
Home	Painel operacional e gráficos
Quiosque	CRUD de barracas e estoque fracionado
AddItem	Cadastro e edição de produtos, upload de imagem
Cart	Marketplace: lista de fornecedores
StoreVitrine / ProductDetail	Catálogo do fornecedor e item
Sacola	Checkout B2B, endereço, pagamento, POST da solicitação
PedidoAcompanhamento	Polling do status até entrega
Configuracoes / Enderecos / FormasPagamento / Cards	Conta, entrega e meios de pagamento
EmpresaVendas / Logistica / Camioneiros	Vendas e frota

Fonte: Elaborado pelos autores.

A sacola agrupa itens por `fornecedorId`. Finalizar dispara uma solicitação por grupo, com taxa de entrega fixa no cliente (`TAXA_ENTREGA`). Sucesso limpa o carrinho e abre o acompanhamento. Produtos creditados no estoque do comprador ocorrem no backend quando o status chega a entregue; o app apenas recarrega a lista.

6 EXPERIÊNCIA DO CLIENTE

Se `perfilUso === 'Cliente'`, a Home delega para `ClienteHomeScreen`. A barra `ClienteTabBar` substitui a barra dourada. Cores de referência: fundo de cabeçalho `#0E2A5C`, ênfase `#F8B125`.

6.1 Navegação inferior



Figura 4 – Abas do perfil cliente

Fonte: Elaborado pelos autores.

A aba Carrinho corresponde à rota interna `Reservas`, para não espalhar rename na pilha. O distintivo numérico é a soma das quantidades em `PurchaseCartContext.itemCount`. Pedidos e o detalhe `PedidoAcompanhamento` marcam a aba Pedidos como ativa.

Tabela 5 – Telas do perfil cliente

Rota	Tela visível	Dados
Home	ClienteHome	fornecedores em destaque
Explorar	Produtos	busca, categorias, vitrines
Reservas	Carrinho	itens locais + checkout (useSacola)
Pedidos	Histórico	GET solicitações-compra
PedidoAcompanhamento	Detalhe	GET por id, polling
Perfil	Meu perfil	atalhos de conta
Configuracoes	Dados do cliente	PUT usuário e empresa
Enderecos	Endereços de entrega	CRUD /api/enderecos
FormasPagamento	Pagamento	formas, PIX e cartões

Fonte: Elaborado pelos autores.

6.2 Compra e carrinho

Explorar e a vitrine `ClienteQuiosqueVitrine` chamam `addItem`. O detalhe `ClienteProductDetail` informa quantidade e respeita estoque informado pela API. A tela Carrinho lista grupos “Vendido por”, stepper, exclusão, endereço (entrega ou retirada) e resumo. Finalizar reutiliza a mesma sequência da Sacola: validação, modal de pagamento e `POST /api/solicitacoes-compra`.

6.3 Pedidos e conta

O histórico filtra em três abas: Realizados (lista completa), Em andamento (status diferente de entregue e cancelada) e Concluídos (entregue). O código exibido é #QS- concatenado ao identificador. O detalhe desenha o stepper Pedido confirmado → Em separação → A caminho → Entregue, usando `pedido.etapas` ou o conjunto padrão do acompanhamento. Perfil reúne dados do cliente, endereço, pagamento, notificações e saída.

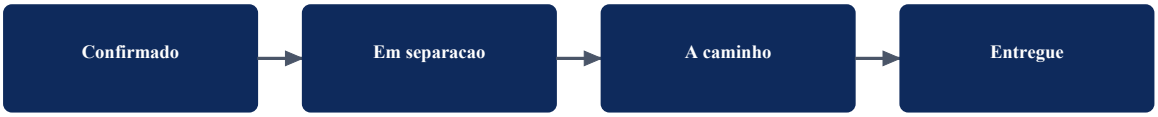


Figura 5 – Etapas de acompanhamento do pedido

Fonte: Elaborado pelos autores.

7 SERVIÇOS E INTEGRAÇÃO COM A API

Os módulos em `src/services` encapsulam verbos HTTP. A tabela 6 resume o que o aplicativo de fato chama. PIX de chaves cadastradas no aparelho não replica tabela remota: a lista é local, mas a forma de pagamento PIX em si é criada em `/api/formas-pagamento`.

Tabela 6 – Endpoints consumidos pelo aplicativo

Método e caminho	Uso no app
POST /api/cadastro	RegisterScreen
POST /api/usuarios/login	Login
POST /api/usuarios/recuperar-senha	ForgotPassword
GET /api/usuarios/me	restauração de sessão
PUT /api/usuarios/{id}	Dados do cliente / configurações
PUT /api/empresas/{id}	nome, CNPJ, telefone
GET/POST /api/produtos	catálogo da empresa e alta
PUT/DELETE /api/produtos/{id}	edição e exclusão
POST /api/produtos/upload	imagem do produto
CRUD /api/barracas	quiosques e estoque da barraca
GET /api/marketplace/fornecedores	home cliente, explorar, cart
GET .../fornecedores/{id}/produtos	vitrine
POST/GET /api/solicitacoes-compra	checkout e histórico
GET .../solicitacoes-compra/{id}	detalhe com polling
CRUD /api/enderecos e GET .../cep/{cep}	entrega
CRUD /api/formas-pagamento	meios no checkout
CRUD /api/cartoes-pagamento	crédito e débito
GET /api/financeiro/resumo	home da empresa
GET /api/notificacoes	sino e modal
GET/POST /api/caminhoneiros e /api/logistica	módulo de frota

Fonte: Elaborado pelos autores.

8 PERSISTÊNCIA NO DISPOSITIVO

O AsyncStorage evita perda ao fechar o Expo Go. Nenhuma dessas chaves substitui o banco: após o POST da solicitação, a fonte da verdade do pedido é a API.

Tabela 7 – Chaves de armazenamento local

Origem	Conteúdo
sessionStorage.ts	token, usuário, expiresAt
perfilStorage.ts	mapa e-mail → PerfilCadastro
purchaseCartStorage.ts	itens do carrinho por empresa
pixChaveService.ts	chaves PIX mascaradas por empresaId
enderecoService (seleção)	id do endereço escolhido no checkout

Fonte: Elaborado pelos autores.

9 INTERFACE GRÁFICA

Há duas paletas convivendo de propósito. A autenticação e o perfil cliente usam cabeçalho navy, superfície `#F3F5F8` ou `#F7F8FB` e botão dourado com texto navy (`authTheme.ts`, `AuthScreenShell`, `AuthBrand`). A operação da empresa mantém o degradê dourado de `theme.ts` e a tab bar amarela. Componentes comuns (`CustomButton`, `CustomInput`) foram alinhados ao formulário claro com ícone navy, inclusive alternância de visibilidade da senha.

A ramificação visual ocorre por `perfilUso` em pontos únicos: Home, barra inferior, StoreVitrine, ProductDetail e PedidoAcompanhamento. Isso evita duplicar a pilha de navegação e reduz risco de rota órfã.

10 CONSIDERAÇÕES FINAIS

O aplicativo cobre o ciclo que o produto se propõe no dispositivo: entrar, escolher como se usa o sistema, operar estoque ou comprar, pagar e acompanhar. A divisão empresa/cliente é de apresentação e de conjunto de telas, não de dois binários. O débito técnico mais visível é o cadastro de cliente ainda reutilizar o documento da empresa no payload de cadastro; qualquer mudança nisso depende de acordo com o backend. Outro ponto de atenção é o atalho admin em código. Como continuidade, cabem testes de regressão no checkout, mascaramento de dados em logs e o retrato oficial da API neste mesmo padrão ABNT, em documento separado.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 14724:
informação e documentação — trabalhos acadêmicos — apresentação. 3. ed. Rio de

Janeiro: ABNT, 2011.

EXPO. Documentation. Disponível em: https://docs.expo.dev. Acesso em: 27 set. 2026.

META PLATFORMS. React Native. Disponível em: https://reactnative.dev. Acesso em: 27 set. 2026.

REACT NAVIGATION. Native stack navigator. Disponível em: https://reactnavigation.org. Acesso em: 27 set. 2026.

STOCK4EVENT. Repositório do aplicativo móvel. Disponível em: https://github.com/LuanSantos26/Mobile. Acesso em: 27 set. 2026.

STOCK4EVENT. Repositório do backend. Disponível em: https://github.com/LuanSantos26/QuickStock-BackEnd. Acesso em: 27 set. 2026.

APÊNDICE A – COMO EXECUTAR O PROJETO

Requisitos: Node.js LTS, Git e o backend Stock4event escutando em 8081. No diretório do aplicativo: `npm install` e `npx expo start`. A tecla `a` abre o emulador Android; `w`, o navegador. No aparelho físico, o Expo Go deve alcançar o IP da máquina de desenvolvimento. Os scripts `npm run android` e `npm run ios` apenas encapsulam o start direcionado.

Este PDF é gerado por `docs/scripts/gerar_pdf_abnt.py`. Para atualizá-lo: `python docs/scripts/gerar_pdf_abnt.py`.